CHAPTER 2

Basics of Mobile Application Development

Content

- Introduction
- Mobile app development technologies
- Type of mobile app
- Mobile App development languages
- Cross platform frameworks

Introduction

- **Mobile application development** is the process to making software for smartphones and digital assistants, most commonly for **Android** and **iOS**.
  - The software can be preinstalled on the device, downloaded from a mobile app store or accessed through a mobile web browser.
- The programming and markup languages used for this kind of software development include Java, Swift, C#, JavaScript and HTML5.

- Mobile app development is rapidly growing.
  - From retail, telecommunications and e-commerce to insurance, healthcare and government, organizations across industries.
- Modern smartphones and tablets are equipped with features such as **Bluetooth**, **Near Field Communication (NFC)**, **GPS**, **gyroscopic sensors**, **cameras**, and many more.
  - Developers can use these features to create apps with technologies such as **Virtual or Augmented Reality**, **barcode scanning**, **location-based services**, and many more. The most successful and popular mobile applications use smartphone features in the best possible way.

Mobile App Development Process

- An effective app development process flow spans over six key phases. Regardless of the size and scope of your project, following this development process will make your enterprise mobile app development initiative a success.

1. Strategy
2. Analysis & Planning
3. UI/UX Design
4. App Development
5. Testing
6. Deployment and Support

MOBILE APP DEVELOPMENT PROCESS
Strategy
Define strategic goals for evolving your idea into a successful app
UI/UX Design
Create seamless and effortless user experiences with a polished design
Testing
Validate your app's quality thoroughly with a series of testing criteria
1
2
3
4
5
6
Analysis & Planning
Identify requirements, define team structure, and prepare product roadmap
App Development
Develop backend, API, and mobile app following agile development process
Deployment & Support
Launch app on the Apple and Google app stores with support for future revisions
INVONTO

Mobile App Development Technologies

- There are **three** mobile apps development technologies including
  - Native apps,
  - Hybrid apps
  - Cross-platform app
- All three development paths hold intrinsic value, but which approach is right for your project?
  - Depending on your business objectives and overall product goals, this decision can make or break the success of your mobile strategy.
  - Deciding to build your mobile product as either a web, native or hybrid app involves a variety of factors for consideration.

- **Native App**
  - The Native apps run on a specific technology dedicated to either the Android or IOS platform. These platforms use their own languages to run their operating systems.
    - Hence the coding differs for Android and IOS.
      - Kotlin or Java is used for building Android mobile app development.
      - Swift or Objective C works for iOS mobile app.
  - Mobile developers expect robust functioning on Android and IOS platforms.
    - User experience a healthy performance as the application is tailor-made to run on both platforms.
    - Native app mobile developers use the resources available on the chosen platform and code them accordingly to work better.

- **Features**
  - Simpler code delivers **Robust performance**
  - Much more **solid functioning** in an offline environment
  - **API usability** is higher
  - Custom made **UI components** enhances the user experience
  - **Preventing bugs** is easier and
  - **Single layouts** for each platform to suit screen-ratios
- **When to use?**
  - Whenever you need only one platform to launch your mobile app.
  - When your application constantly depends on Camera, GPS, and more
  - Developing 3D games or games based on lots of animations.
  - When you want to give your audience the best experience and especially when your company is launching the app for the first time.

- **Tools to use:-**

iOS:

Android:

- **Hybrid app development**
  - Hybrid mobile app development uses the features of a native app and a web app.
    - Mobile developers build a hybrid mobile app using well-known languages like JavaScript, HTML, and CSS.
    - A hybrid app is about combining mobile and web elements to develop a product to target different platforms quickly.
  - Making hybrid apps consists of creating backend codes based on the native shell (Andriod or IOS) which then be uploaded to Google Play or Apple App Store.

- **Features**
  - Hiring hybrid Mobile app developers is relatively cheap as apps work on many different platforms
  - App reaches the market faster as the backend codes remain more or less the same for all mobile variants.
  - Hybrid apps are much easier to re-code for releasing new updates.
- **When to use?**
  - You have a bigger audience who will use your app on many platforms – mobile and web
  - When you develop simple apps to connect with your customers without major animations and other complex features
  - When you are time-bound and want to create an application to launch on multiple platforms
  - Test your idea in Minimum Viable Product (MVP)

- Tools to use

- **Cross-platform app development**
  - Cross-platform app development shares the same codebase as hybrid development.
    - Mobile developers use the same native elements to create the app, but they differ in providing a great user experience with different UI.
  - Cross-platform apps are more popular, thanks to Flutter and React Native.

- **Features**
  - Building apps based on cross-platform apps help to save time and money.
  - You may add features quickly as the code base requires only a little tweaking.
  - UI performance matches with Native apps as it shares a similar codebase.
- **When to use?**
  - Initially low on budget, cross-platform is the ideal choice.
  - If you are unaware of which groups to target, launch your app to target both audiences.
  - Your app is not complex and does not require extensive coding.

- Tool to use:

Comparison Table
(Native vs Hybrid vs Cross-Platform)

App Type	Native	Hybrid	Cross-platform
Codebase	separate codebases – one per platform	single codebase with potential platform-specific abilities	single codebase with potential platform-specific abilities
Libraries	Not much dependency on open-source libraries and platforms	Highly dependent on different libraries and frameworks	Highly dependent on different libraries and frameworks
Debugging	Native debugging tools	Native + web development debugging tools	Depends on the framework*
Tools	– XCode – AppCode – Android Studio	– Ionic – Apache Cordova – Visual Studio	– React Native – Xamarin – Flutter

Pros	– Full access to device's/OS' features – Powerful performance – Native UI (updated along with the OS) – Efficient App – Running – High-quality functionality and UX – Access to all native APIs and the platform-specific functionality	– Lower development costs – Different OS support – Code reuse – Cost-effective development – Big customization capabilities	– Different OS support – The UI is almost as quick as native. – Code reuse – Development at a low cost
Cons	– No multi-platform support – High dev costs if different OS support is needed – No code reuse	– Slower performance – OS features are restricted – No interaction with other native apps	– Slower performance – Limited access to OS feature – Poor interaction with other native apps

- **Mobile web apps** are a lightweight paradigm where you open a URL in your device's browser, but they look and feel like a regular mobile app.
  - Mobile websites are not delivered as an installed app on the device, but they are an important paradigm to consider while building for a mobile device.
  - Developers can use two web site programming paradigms to create mobile web apps:
    - **Responsive web programming** is a style of web site programming where the web site layout organizes itself to suit the device's form factor.
    - **Progressive web programming** creates progressive web apps that are normal websites that appear like a traditional mobile app when navigated to from a browser on a mobile device.

Types of Mobile App

- Mobile applications come in many shapes and sizes. Here are the most popular types of mobile apps to help you understand the current trends in the mobile landscape.
  - **Gaming apps**
  - **Business or productivity apps**
  - **Educational apps**
  - **Lifestyle apps**
  - **M-commerce apps**
  - **Entertainment apps**
  - **Utility apps**
  - **Travel apps**

Mobile App Development Programming Languages

- When considering the programming languages, frameworks, and SDKs for mobile apps, you have to consider the frontend (UI) development environment but also be aware of the backend (server-side) development environment.
- The developers who are coding the frontend are often not the ones who are coding the backend, but they do usually work with each other (they are in enterprises anyway).

Developing mobile apps for the iOS platform

- The iOS platform is a proprietary platform made by Apple.
  - The iOS platform consists is available for phone devices (iPhone) and tablet devices (iPad).
- You can develop apps for the iOS platform, and then target the same app to both an iPhone and an iPad.
  - While developing the app, you have to account for the real estate constraints that are available on each of the devices.
  - iOS apps can be installed on your iPhone or iPad from the Apple app store.
  - For building apps for iOS, you must have an Apple developer account and the Xcode IDE on a Mac computer. You cannot effectively build and debug on a Windows machine alone.

- **Objective-C**
  - *Objective-C* was the first language supported by Apple for developing iOS mobile apps.
    - It is an **objected-oriented (OO) language** that derives the language syntax from C and the OO-ness from SmallTalk.
    - One of the common criticisms of the language is that its syntax seems clunky being verbose and the square brackets are hard to debug. It's a stable and mature language, however, as it has had several years of wide usage.
  - Since Apple introduced Swift, the popularity of Objective-C has declined considerably for new iOS mobile development.

- **Swift**
  - Apple introduced *Swift* in 2014 as a language specification, and made it available for mobile app development on Xcode in 2015.
    - After a major overhaul of the Swift language in 2016 (v 3.0), Swift has surpassed Objective-C as the language to write native iOS apps.
  - Though Swift and Objective-C can coexist, that is libraries written in Objective-C and Objective-C utilities can be used in Swift, Apple is making it very obvious that Swift is the new default choice for developing iOS apps.
  - Swift is an easier, simpler, and a more compact language compared to Objective-C. Objective-C developers should not have any trouble moving over to Swift.

Objective-C

```
const int count = 10;
double price = 23.55;

NSString *firstMessage = @"Swift is awesome. ";
NSString *secondMessage = @"What do you think?";
NSString *message = [NSString stringWithFormat:@"%@%@", firstMessage, secondMessag
e];

NSLog(@"%@", message);
```

Swift

```
let count = 10
var price = 23.55

let firstMessage = "Swift is awesome. "
let secondMessage = "What do you think?"
var message = firstMessage + secondMessage

print(message)
```

Developing mobile apps for the Android platform

- Android is an open source platform that is primarily developed and promoted by **Google**. Google promotes its own mobile device brand, Pixel (and the earlier brand, Nexus).
  - However, there are large number of manufacturers like Samsung, Huawei, Xiaomi, and Oppo that sell their own branded phones and tablets that are powered by Android and derivatives of the Android OS like CyanogenMod and MIUI.
- Android devices come in a large number of form factors, from phones to tablets, because of a variety of manufacturers with multiple models that address different user preferences.

- For building Android apps, you need the **Android development toolkit** that comes with the required **SDK**, **debuggers**, and **emulators**.
  - For an IDE, the *Android Studio* is by far the most popular, but there are other equally popular IDEs like *Apache Netbeans* and *IntelliJ IDEA*.
  - You can have the **development environment on any OS** (Mac, Windows or Linux).
- Android uses ***Gradle*** for the build system, and Android Studio comes with Code templates and other performance assessment and enhancement tools.

- Java
  - Java was the default language to write Android apps since the Android platform was introduced in 2008.
  - Java is an object-oriented programming language that was originally developed by Sun Microsystems in 1995 (now, it is owned by Oracle).
  - It was a very popular language as a pure object-oriented language (as compared to C++) and was quickly adopted by the Android platform.
  - Java compiles to “bytecode” that is interpreted at runtime by the underlying Java Virtual Machine (JVM) that’s running on the OS.
    - You write the mobile apps in Java and program against the Android SDK.
  - And yet, as of 2019, Kotlin is now the preferred language for Android development, as per Google

- Kotlin
  - In 2017 Google announced that it will support Kotlin as an alternative first-class language for Android programming.
    - Kotlin is interoperable with Java, and all Java libraries can be called from Kotlin.
  - Loosely speaking, Kotlin is a tidier form of Java. The learning curve from Java to Kotlin is smooth. At an execution level, Kotlin compiles to Java Bytecode.

Developing mobile apps for both the iOS and Android platforms

- With native app building approaches, you have to maintain one code base per platform. The code repository size is proportional to the platforms that need to be supported.
- You often need to develop for both the platforms, but you don't need the native language capabilities.
  - Luckily, there are technologies that allow you to write in one language or framework and target the app for both the platforms, which means that developers that are not familiar with Java and Swift but are experts in other technologies like Web or C# can use their skills to develop apps for Android and iOS.
- Various open source communities or companies have come up with frameworks that have an affinity to their developer base to develop for Android and iOS.
  - These frameworks come with their own ecosystem of tools and IDEs that make it convenient for the developers familiar with those frameworks.

- **C# and Xamarin**
  - C# is an object-oriented programming language that was developed by Microsoft.
    - The *Xamarin* framework (acquired by Microsoft) allows you to program in C# against the .NET framework. The .NET framework is implemented on the iOS platform using an open source implementation called mono.
  - The popular IDEs for writing C# and Xamarin code is **Visual Studio Code** by Microsoft. The C# code is cross-compiled and runs natively on the iOS or Android device.
    - This allows for a no-lag execution that comes very close to native development. There are special extensions called **Xamarin.iOS** and **Xamarin.Android** that you can use to access iOS and Android native capabilities that can be called from C#.
    - For iOS, you need XCode on a Mac machine to build the installable iOS app.

- **JavaScript and React Native**
  - React Native was released in 2015 by Facebook.
  - React Native uses **JavaScript** as a programming language for writing mobile apps.
    - There is no HTML used in writing React Native apps.
  - This code is then interpreted at runtime and executed using a “bridge” for accessing the device’s native SDK capabilities.
  - React Native apps use the platform native UI library to render the UI components, which makes the UI truly native.
    - React Native has become very popular because the learning curve for JavaScript is very low.

- **Dart and Flutter**
  - Though Dart first was made public by Google in 2011, 2017 was the first stable release. *Dart* is an object-oriented programming language.
    - You use Google's *Flutter framework* to write mobile apps for iOS.
  - The most popular IDEs are Android Studio, IntelliJ, and Visual Studio Code (VSCode).
  - One of the unique capabilities of Flutter is that it comes with its own UI widget framework library that is based on Google Material design as well as iOS-like UI widgets.

Developing mobile apps for the Web platform

- Though not an installed app in the true sense, it's important to consider the UIs that are delivered by navigating to a web URL (web sites).
  - Delivering web sites has some value for certain use cases, such as an app for incidental or short-lived event, or for capturing small amounts of information like name and address without forcing a user to install an app.
  - With the advent of progressive web apps (PWAs) the user experience for apps and websites are indistinguishable.
- These mobile-optimized websites are written using web technologies (HTML, JavaScript, and CSS) and typically a UI framework.

- Mobile-optimized websites
  - Using any of the frameworks like Material, Bootstrap, or Foundation, you can deliver a responsive web site. Responsive web design has layouts that adapt to the available real-estate by giving a native experience.
    - For example a website might be a 5-column layout in a laptop browser, but that website will render in a 2-column layout when viewed from a browser on a mobile device in portrait mode.
    - The two primary differences between a mobile-optimized website and viewing a regular website on a mobile phone is that you should not have to zoom in to view the content and there is no horizontal scrolling.

- **Progressive web apps (PWAs)**
  - *Progressive web apps* are actually web sites that render like a native app. The browsers have also evolved to detect that it's a PWA and will remove the surrounding browser to give the web content the full device real estate.
  - When you add a shortcut for this web site, it creates an icon on the mobile home screen, so that you can launch it just like a regular app. The browser also caches specified content locally on the device so that you have a fast and reliable load time and a progressive offline experience as well. This is achieved by using:
    - A service worker that works in the background to pre-cache the content
    - Responsive design for optimal rendering
    - A web app manifest to notify the browser that it's installable

Cross-Platform Development

- It is an approach that allows developers to build a mobile solution that is simultaneously **compatible with several operating systems and platforms** (Android, iOS, Windows).
- **Cross-platform development** aims at creating a single application that runs identically on several platforms.

Native vs Cross-platform Apps

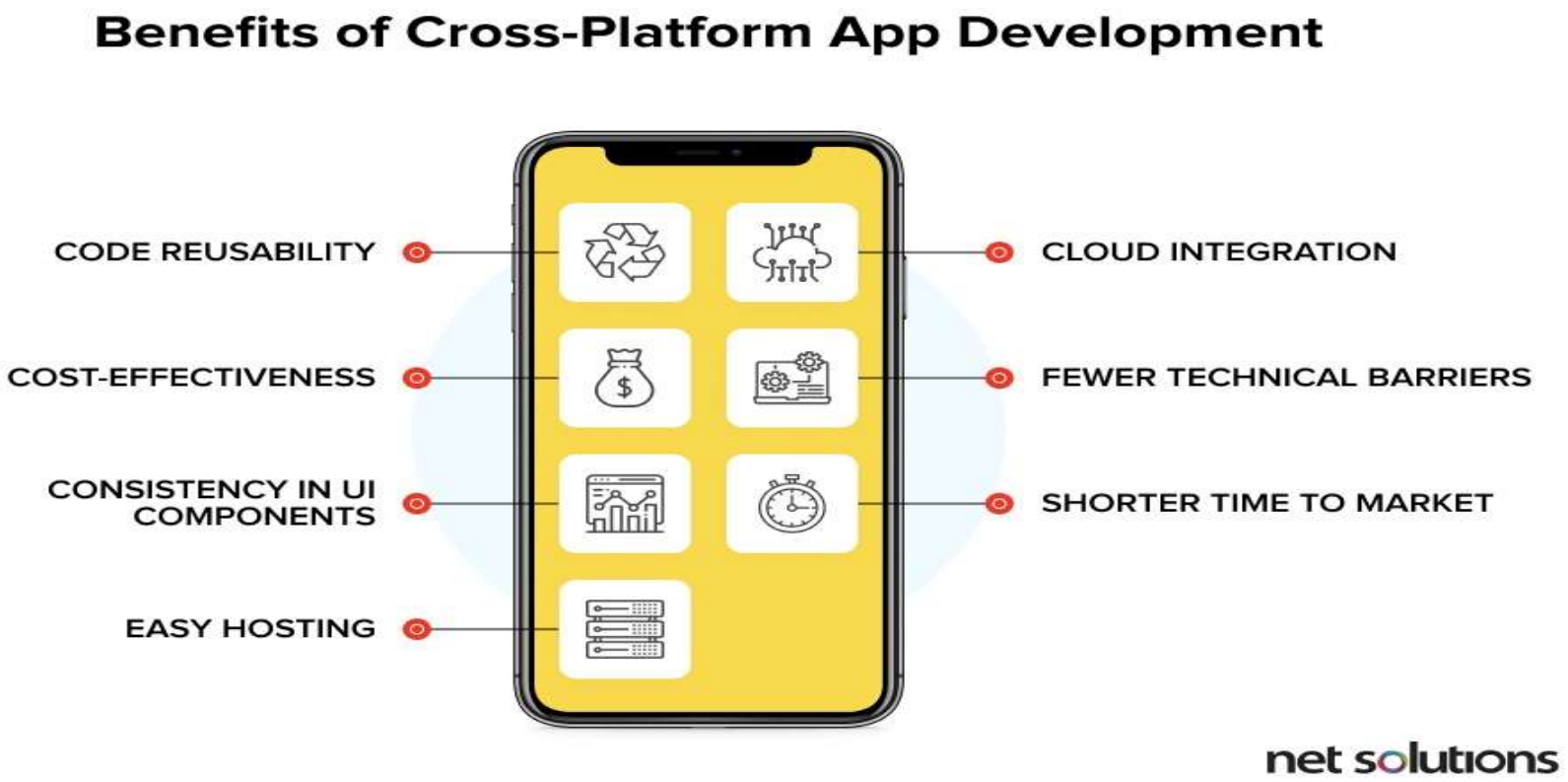
Parameter	Native Apps	Cross Platform Apps
Cost	High cost of development	Relatively low cost of development
Code Usability	Works for a single platform	Single code can be used on multiple platforms for an easy portability
Device Access	Platform SDK ensures access to device's API without any hindrance	No assured access to all device APIs
UI Consistency	Consistent with the UI components of the device	Limited consistency with the UI components of the device
Performance	Seamless performance, given the app is developed for the device's OS	High on performance, but lags and hardware compatibility issues are not uncommon

Challenges & Benefits

- A few years ago, cross-platform app development was constrained to making simple mobile apps and games.
- Over time, emerging technologies have made cross-platform development more adaptable, powerful, and flexible than before.

- **Challenges**
  - Performance hiccups because of inconsistent communication between the native and non-native components of gadgets
  - Cross-platform app developers have difficulty maintaining cross-compliance of apps across devices and operating systems
  - Performance-related glitches can lead to poor user experience
  - If a business app manages part of a corporation and stores the user’s data, then going for cross-platform apps is not always a good idea due to security concerns
- However, these challenges are minimal when compared to the benefits of cross-platform app development.

Benefits


Benefits of Cross-Platform App Development
CODE REUSABILITY
COST-EFFECTIVENESS
CONSISTENCY IN UI COMPONENTS
EASY HOSTING
CLOUD INTEGRATION
FEWER TECHNICAL BARRIERS
SHORTER TIME TO MARKET
net solutions

- **Maximum Exposure to the Target Audience**
  - Leveraging a mobile cross-platform development approach enables a company to build an app and deploy it over various platforms, including the web. This means that, by building a single app, one can target both iOS and Android platforms. Thus, maximizing their reach.
- **Reduced Development Cost**
  - The cross-platform app development is based on the concept ‘write once, run everywhere. Reusable codes and Agile app development processes and tools can lessen the cost of development. Therefore, in order to improve a business on multiple platforms in a cost-effective way, there is no other alternative to cross-platform apps.

- **Easier Maintenance & Deployment**
  - In cross-platform app development, a single and a universal app is compatible for running on all platforms. This makes it easier to maintain and deploy code as changes are made. Updates can promptly be synced over all platforms and devices, thus saving time and money. Moreover, if a bug is found in the common codebase, it only needs to be fixed once. In this way, developers can save a lot of time and money.
- **Quicker Development Process**
  - The quick development process is another win-win situation when it comes to developing cross-platform apps. Single source code for multiple platforms can help reduce the development efforts by 50 to 80%. The process helps to develop a feature-rich business app in less time. The team of developers can meet the expected deadlines in cross-platform app development.

- **Reusable Code**
  - Another good thing about this platform is that the code can be used again and again. Instead of developers developing new codes for every platform, a single code can be reused. This saves time as well as resources because it eliminates repetition in the task of creating codes.
- **Easy Integration with Cloud**
  - Cross-platform mobile apps are totally compatible and can take advantage of various plugins integrated with the cloud settings. In other words, the single source code is coordinated with various plug-ins and extensions to enhance the app’s scalability and functionality.

- **Faster Time-to-Market and Customization**
  - As mentioned above ‘write once, run everywhere is the concept that is followed while building cross-platform app development. This allows app developers to reduce Time-to-Market (TTM) with quick deployment.
  - Also, if the app needs to be transformed or customized, it is easy for the developers to do minor changes in a single code. This helps to deliver products more swiftly than the competitors by improving customer engagement.
- **Uniform Design**
  - Users can recognize user interface (UI) elements, and foresee their interactions over various platforms. Therefore, User Experience (UX) is an important thing to consider for any app or software.
  - It is hard to sync the various development projects while developing multiple apps. Cross-platform mobile development tools allow developers and designers to build a uniform user experience that app users can enjoy.

Open-source Cross-Platform App Development Frameworks Based on Programming Languages

Programming Language	Framework
JavaScript	React Native, Cordova, NativeScript, Appcelerator
Dart	Flutter
C#	Xamarin
Java	Codename One
Python	Kivy, BeeWare
Ruby	RubyMotion

Best Cross Platform App Development Frameworks

- There are several cross-platform app development frameworks out there, each with its own set of pros and cons.
- The most competitive and top-performing frameworks available today are described below
  - Xamarin
  - React Native
  - Flutter

- **Xamarin**
  - Xamarin is a leading free, open-source platform for building hybrid mobile apps.
  - Xamarin-based solutions are compatible with any mobile platform and provide the same performance and user experience as native solutions.
- This popular Microsoft cross-platform mobile app framework relies on the C# programming language complemented with .NET framework and is available under the MIT license as part of Visual Studio

- Xamarin Pros and Cons
  - Pros of Xamarin
    - Xamarin app development uses C# for coding. This means that it works seamlessly on an array of platforms (including Android and iOS)
    - Xamarin has a strong community of over 60,000 contributors from more than 3,700 companies
    - Xamarin allows for sharing more than 75% of code across platforms, for “write once, run anywhere” ease
    - Xamarin consists of a single tech stack for faster development

- Cons of Xamarin
  - Xamarin is expensive for enterprises. Xamarin is a framework that comes free for individuals and startups, however, enterprises are required to buy a license for Microsoft’s Visual Studio
  - Xamarin is not recommended for apps that demand heavy graphics because each platform has a different method for visually laying out screens. A UX/UI-rich application is advised to be implemented natively
  - Xamarin also offers limited access to certain important libraries that the app developers need for mobile app development. Also, since the core of its user-interface creation is not mobile, creating the UI is time-consuming

- Apps Made with Xamarin Cross-Platform App Framework
  - Fox Sports
  - Alaska Airlines
  - HCL
  - American Cancer Society
  - BBC Good Food

- **React Native**
  - React Native is one of the top cross-platform mobile development frameworks.
    - It is based on React — a best-in-class JavaScript library for creating user interfaces — and targets mobile platforms.
  - The great thing about React Native is that you can use a universal JavaScript language as well as Java, Swift, or C — languages specific for native platforms to build cross-platform solutions.
    - Thus, this framework can convert the source code into native elements, contributing to native user experience.
  - Web developers can easily use React Native to create mobile apps without obtaining new programming skills.

- **Pros of React Native**
  - Up to 80% of a codebase can be shared across platforms, depending on the app's complexity. Code reusability significantly speeds the development process
  - React allows for previewing results right away and offers ready-to-apply elements, thus shortening the developing time considerably
  - The Hot Reloading feature enables developers to see changes made in code within seconds, not minutes, like when using native technologies
  - React Native focuses on UI to a great extent rendering a highly responsive interface
  - It also provides access to certain useful native functionalities like accelerometer and camera. This allows for rendering a high-quality, native-like user interface

- **Cons of React Native**
  - React Native is not fully a cross-platform app framework. To use some functions as a camera or accelerometer there is still a need to use native components meaning there will be a separate code for Android and iOS
  - Since the framework is not built in conjunction with iOS or Android, it lags behind the native platforms at times. This is one of the reasons that led Udacity to stop investing in React Native for new features
  - React Native lacks consistency when it comes to releasing updates
  - React Native improves the speed of development, but also increases the duration of the debugging process, especially on Android

- Apps Made with React Native Cross-Platform App Framework
  - Facebook
  - Instagram
  - Bloomberg
  - Pinterest
  - Skype
  - Tesla

- **Flutter**
  - Flutter is an open source and free cross-platform framework for creating native interfaces for Android and iOS.
    - Remember, Flutter is a cross-platform app framework maintained by **Google**, the very same organization that develops the **Android Native Framework**.
  - Applications that are programmed in Dart language act almost identically on each mobile operating platform (Android & iOS) and they have similar efficiency to their native solutions. Moreover, apps made with Flutter are indeed native

- Pros of Flutter
  - “Hot reloading” feature enables developers to see changes made in code within seconds as opposed to minutes
  - Flutter is an ideal framework for MVP development. Instead of spending extra money and time on two separate apps, a Flutter mobile application that looks native on both Android and iOS can be built quickly
  - Flutter is based on Dart, an object-oriented programming language that developers have found rather easy to acquire the skill for
  - Flutter has a full set of widgets in Google’s Material Design and in Apple’s style with the Cupertino pack
  - Many ready-made solutions for native Android and iOS apps enable developers to work with Continuous Integration platforms like Travis and Jenkins

- **Cons of Flutter**
  - There is limited TV support with apps built on the Flutter framework (i.e., Flutter offers no support for Android TV and Apple TV)
  - Though by virtue of being developed by Google, there are several libraries with ready-to-implement functionalities, however, Flutter still lacks when compared to native development
  - Since Flutter-enabled apps use built-in widgets and not platform widgets, the app’s size is usually bigger. Currently, the smallest possible app made with Flutter can weigh no less than 4MB

- Apps Made with Flutter Cross-Platform App Framework
  - Alibaba
  - Google
  - Google Ads
  - Tencent
  - ebay

Questions?

Thank You